Școala: Liceul Național de Informatică Arad

LUCRARE DE ATESTAT

DISCIPLINA INFORMATICĂ

Candidat,

Ciobanu David

Coordonator,

Prof. Dascal Ioan

2026

Tema lucrării

Aplicație pentru Android și Windows numită Control Bot.

Cuprins

Tema lucrării ..... pag. 2
I. Introducere / Argument ..... pag. 4
I.1 Justificarea alegerii temei ..... pag. 4
I.2 Echipamentul și aplicațiile necesare ..... pag. 5
II. Partea teoretică ..... pag. 7
II.1 Limbajul de programare Java ..... pag. 7
II.2 Platforma Android și mediul Android Studio ..... pag. 8
II.3 Comunicarea în rețea: Sockets TCP/IP ..... pag. 9
III. Descrierea părții practice ..... pag. 11
III.1 Crearea aplicației pentru Android ..... pag. 11
III.1.1 Interfața principală (conectare) ..... pag. 11
III.1.2 Interfața alegerii modului de control ..... pag. 13
III.1.3 Interfața „Text” ..... pag. 13
III.1.4 Interfața „Mouse” ..... pag. 14
III.1.5 Interfața „Keyboard” ..... pag. 15
III.1.6 Comunicarea între telefon și calculator ..... pag. 16
III.2 Crearea aplicației pentru Windows ..... pag. 17
IV. Concluzii ..... pag. 20
IV.1 Realizări și rezultate obținute ..... pag. 20
IV.2 Dificultăți întâmpinate și lecții învățate ..... pag. 20
IV.3 Perspective de dezvoltare ulterioară ..... pag. 21
V. Bibliografie ..... pag. 23

I. Introducere / Argument

I.1 Justificarea alegerii temei

Opțiunea pentru dezvoltarea aplicației Control Bot a fost determinată de o convergență de interese tehnice și de o necesitate practică recunoscută în mediul tehnologic contemporan. În primul rând, alegerea limbajului de programare Java ca fundație a întregului proiect nu este una întâmplătoare, ci rezultatul unei analize atente a avantajelor pe care acest limbaj le oferă în contextul dezvoltării de aplicații cu interfețe grafice și comunicare în rețea. Java se distinge prin portabilitatea sa excepțională — principiul „write once, run anywhere" —, prin arhitectura sa orientată pe obiecte, care facilitează modularitatea și reutilizarea codului, și prin ecosistemul său matur de biblioteci și framework-uri. Aceste caracteristici îl transformă într-un instrument ideal pentru realizarea de aplicații care trebuie să comunice între platforme diferite, cum este cazul proiectului de față.

Alegerea temei a fost influențată și de dorința de a explora domeniul Internet of Things (IoT), în care dispozitive diverse comunică și interacționează în timp real. Control Bot poate fi considerat un prim pas în această direcție, demonstrând că un telefon mobil poate servi ca interfață de control pentru un calculator personal, transformându-se într-un dispozitiv de tip IoT. Această perspectivă adaugă o dimensiune suplimentară proiectului, plasându-l într-un context tehnologic mai larg și mai relevant pentru tendințele actuale din industria software.

Un alt factor determinant în alegerea temei a fost realitatea pieței software actuale, în care interoperabilitatea între dispozitive a devenit o exigență fundamentală. Utilizatorii moderni nu mai percep calculatorul și telefonul ca entități izolate, ci ca noduri ale unui ecosistem integrat, între care trebuie să existe o comunicare fluentă și o controlare reciprocă. În acest context, dezvoltarea unei soluții care permite controlarea de la distanță a unui calculator prin intermediul unui telefon Android răspunde unei nevoi reale și actuale. Mai mult, limitarea funcționării la mediul Windows ar fi redus semnificativ utilitatea și atractivitatea proiectului, având în vedere dominanța dispozitivelor mobile în viața cotidiană.

Proiectul Control Bot îmbină, așadar, provocările programării în rețea cu cele ale dezvoltării de interfețe grafice pe mobil, oferind un cadru de învățare complex și relevant. El necesită înțelegerea profundă a modelului client-server, a protocolului TCP/IP, a gestionării concurente a mai multor conexiuni și a emulării dispozitivelor de intrare (tastatură și mouse), toate acestea fiind competențe esențiale pentru un programator modern. Dezvoltarea acestui proiect a reprezentat o oportunitate valoroasă de a consolida cunoștințele teoretice prin aplicarea lor într-un context practic autentic, cu provocări reale de design, implementare și testare.

I.2 Echipamentul și aplicațiile necesare pentru realizarea proiectului

Echipament hardware. Dezvoltarea aplicației Control Bot a necesitat un echipament hardware capabil să susțină simultan mediile de dezvoltare pentru două platforme distincte. S-a utilizat un laptop echipat cu 32 GB de memorie RAM, unități de stocare SSD de mare viteză și un procesor cu performanțe adecvate, configurație care a permis rularea mașinilor virtuale Android (emulatoare) în paralel cu mediul de dezvoltare IntelliJ IDEA, fără degradarea performanțelor. Această cerință de resurse este justificată de faptul că Android Studio consumă semnificativ memorie în timpul compilării și rulării aplicațiilor, iar testarea simultană a componentei client și a celei de server necesită un mediu de execuție stabil. Pentru testarea în condiții reale a aplicației mobile, s-a utilizat un telefon fizic cu sistemul de operare Android, conectat la aceeași rețea locală cu calculatorul-gazdă, esențial pentru validarea funcționalității de comunicare în rețea.

Accesul la un echipament hardware performant a fost esențial nu doar pentru dezvoltare, ci și pentru procesul de depanare. Rularea simultană a Android Studio cu emulatorul activ, a IntelliJ IDEA cu serverul în execuție și a instrumentelor de monitorizare a rețelei (de exemplu, Wireshark pentru analiza pachetelor TCP) solicită semnificativ resursele sistemului. Configurația cu 32 GB RAM și SSD-uri de mare viteză a permis această utilizare simultană fără degradarea timpului de răspuns, aspect critic în timpul sesiunilor intensive de depanare în care era necesară observarea în timp real a traficului de rețea.

Mediul de dezvoltare pentru Windows. Pentru componenta de server destinată sistemului de operare Windows, s-a ales IntelliJ IDEA, un mediu de dezvoltare integrat recunoscut pentru suportul său excelent pentru limbajul Java. IntelliJ oferă funcționalități avansate precum completarea inteligentă a codului, refactoring automat, depanare integrată și gestionarea eficientă a proiectelor Maven sau Gradle. În acest mediu, s-a utilizat JDK 25 (Java Development Kit) în configurație fără sistem de build extern, compilarea și execuția fiind gestionate direct din IDE.

Mediul de dezvoltare pentru Android. Pentru componenta client destinată sistemului Android, s-a utilizat Android Studio, un IDE dezvoltat de Google pe baza platformei IntelliJ, specializat pentru construirea aplicațiilor mobile. Android Studio integrează SDK-ul Android, emulatorul de dispozitive, instrumente de profilare a performanței și un editor vizual de interfețe. În cadrul acestui proiect, s-a utilizat Android SDK 34, cu sistem de build Gradle versiunea 9.0.0, configurație care asigură compatibilitatea cu versiunile recente ale sistemului de operare Android și accesul la cele mai noi API-uri ale platformei.

II. Partea teoretică

II.1 Limbajul de programare Java

Java este un limbaj de programare de nivel înalt, orientat pe obiecte, dezvoltat inițial de Sun Microsystems în anul 1995 și administrat în prezent de Oracle Corporation. De-a lungul celor peste trei decenii de existență, Java s-a impus ca unul dintre cele mai utilizate și influente limbaje de programare la nivel global, fiind adoptat atât în mediul academic, cât și în cel industrial, în domenii care variază de la dezvoltarea aplicațiilor de întreprindere până la sisteme embedate și aplicații mobile.

Portabilitatea și mașina virtuală Java (JVM). Principiul fundamental care stă la baza succesului Java este portabilitatea codului compilat. Spre deosebire de limbajele compilate nativ, care generează cod executabil specific unei arhitecturi hardware, Java compilează codul sursă în bytecode — un set de instrucțiuni intermediare care sunt interpretate și executate de Java Virtual Machine (JVM). Această arhitectură permite ca un program Java compilat să ruleze pe orice platformă pentru care există o implementare a JVM, fără a necesita recompilare, principiu sintetizat în celebra deviză „write once, run anywhere”. În contextul proiectului Control Bot, această caracteristică este esențială, deoarece componenta de server, dezvoltată în Java, poate fi executată pe orice sistem de operare care dispune de o JVM compatibilă, deși implementarea curentă vizează exclusiv platforma Windows.

În plus față de portabilitate, JVM oferă și optimizări de performanță în timpul execuției prin intermediul compilatorului Just-In-Time (JIT), care translatează bytecode-ul în cod nativ optimizat pentru platforma gazdă. Această caracteristică permite aplicațiilor Java să atingă performanțe comparabile cu cele ale aplicațiilor compilate nativ, contrazicând prejudecata conform căreia Java ar fi un limbaj lent. În contextul Control Bot, aceasta înseamnă că emularea tastaturii și a mouse-ului se realizează cu o latență neglijabilă, asigurând o experiență de utilizare fluidă.

Programarea orientată pe obiecte. Java este un limbaj strict orientat pe obiecte, cu excepția tipurilor primitive de date. Principiile OOP — încapsularea, moștenirea, polimorfismul și abstractizarea — sunt implementate în mod nativ și constituie fundația arhitecturală a oricărei aplicații Java. În cadrul proiectului Control Bot, paradigma orientată pe obiecte a facilitat organizarea codului în clase distincte pentru gestionarea comunicațiilor (SocketManager, TextCommunication), pentru definirea activităților interfeței grafice și pentru implementarea logicii de server, asigurând astfel modularitatea și mentenabilitatea codului.

Gestionarea automată a memoriei. Java dispune de un mecanism automat de colectare a gunoiului (garbage collector), care eliberează memoria ocupată de obiectele ce nu mai sunt referențiate de program. Această funcționalitate elimină necesitatea gestiunii manuale a memoriei, reducând semnificativ riscul erorilor de tip memory leak sau dangling pointer, frecvente în limbajele cu gestiune manuală precum C sau C++. În contextul aplicației Control Bot, acest mecanism este deosebit de valoros, deoarece comunicarea prin socket-uri implică crearea și distrugerea frecventă a obiectelor de tip buffer și flux de date, operațiuni care, într-un limbaj cu gestiune manuală, ar necesita o atenție sporită la eliberarea resurselor.

Biblioteci și API-uri standard. Ecosistemul Java oferă un set extensiv de biblioteci standard (Java Class Library), care acoperă domenii precum comunicarea în rețea (pachetele java.net și java.io), procesarea concurentă (java.util.concurrent), interfețe grafice și conectivitate la baze de date. În dezvoltarea aplicației Control Bot, pachetele java.net și java.io au fost utilizate intensiv pentru implementarea comunicației prin socket-uri TCP, iar pachetul java.awt.robot a fost esențial pentru emularea tastaturii și a mouse-ului pe platforma Windows.

II.2 Platforma Android și mediul Android Studio

Android este un sistem de operare mobil bazat pe nucleul Linux, dezvoltat inițial de Android Inc. și achiziționat de Google în 2005. În prezent, Android deține o cotă de peste 70% din piața sistemelor de operare mobile la nivel global, fiind platforma dominantă pentru dispozitivele smartphone și tabletă. Arhitectura platformei Android este organizată pe mai multe straturi: nucleul Linux pentru gestionarea hardware-lui, biblioteci native, un mediu de execuție (Android Runtime — ART) și un framework de aplicații care expune API-uri dezvoltatorilor.

Activități și ciclul de viață. În modelul de dezvoltare Android, o aplicație este compusă din una sau mai multe activități (Activities), fiecare activitate reprezentând un ecran cu care utilizatorul interacționează. Fiecare activitate parcurge un ciclu de viață bine definit, gestionat de sistemul de operare, care include stările created, started, resumed, paused, stopped și destroyed. Înțelegerea acestui ciclu este critică pentru gestionarea corectă a resurselor — de exemplu, eliberarea conexiunilor de rețea atunci când activitatea trece în starea paused sau destroyed. În cadrul aplicației Control Bot, fiecare interfață (conectare, selecție mod de control, Text, Mouse, Keyboard) este implementată ca o activitate distinctă, cu gestionare explicită a resurselor de comunicare în funcție de starea ciclului de viață.

Android SDK. Software Development Kit (SDK) Android oferă instrumentele și API-urile necesare pentru dezvoltarea aplicațiilor. Versiunea SDK 34, utilizată în acest proiect, corespunde Android 14 și oferă acces la funcționalitățile cele mai recente ale platformei, inclusiv gestionarea

permisiunilor granulare pentru accesul la rețea și optimizări de performanță. Gradle, sistemul de build utilizat de Android Studio (versiunea 9.0.0 în acest proiect), automatizează procesul de compilare, gestionare a dependențelor și generare a pachetelor APK.

Android Studio. Android Studio este mediul de dezvoltare integrat oficial pentru platforma Android, construit pe baza IntelliJ IDEA de JetBrains. Acesta oferă un editor vizual de interfețe (Layout Editor), un emulator de dispozitive integrat, instrumente de profilare a performanței și de depanare avansată. În contextul proiectului Control Bot, Android Studio a facilitat atât proiectarea vizuală a celor cinci activități ale aplicației, cât și implementarea logicii de comunicare prin socket-uri, oferind funcționalități de completare inteligentă a codului și refactoring rapid.

II.3 Comunicarea în rețea: Sockets TCP/IP

Modelul client-server. Comunicarea între aplicațiile Control Bot se bazează pe arhitectura client-server, un model fundamental în programarea în rețea. În acest model, serverul este un proces care așteaptă și acceptă conexiuni de la clienți, în timp ce clientul este procesul care inițiază conexiunea către server. Serverul rulează pe calculatorul Windows și ascultă pe un port specific, iar clientul rulează pe dispozitivul Android și se conectează la adresa IP a serverului pe portul respectiv. Separarea rolurilor între client și server permite o arhitectură clară, în care fiecare componentă are responsabilități bine definite: clientul generează comenzile de control, iar serverul le execută.

Protocolul TCP. Transmission Control Protocol (TCP) este un protocol de transport orientat pe conexiune, care garantează livrarea fiabilă și ordonată a datelor între două entități de rețea. Spre deosebire de UDP, care oferă o comunicare fără conexiune și fără garanții de livrare, TCP stabilește o conexiune prin procesul de three-way handshake, asigură retransmiterea pachetelor pierdute și menține ordinea mesajelor. Aceste caracteristici sunt esențiale pentru aplicația Control Bot, unde fiecare comandă emisă de telefon (apăsarea unei taste, mișcarea mouse-ului, un click) trebuie să ajungă la server în ordinea corectă și fără pierderi, pentru a asigura un control precis și previzibil al calculatorului.

Socket-uri în Java. Java oferă suport nativ pentru comunicarea prin socket-uri TCP prin intermediul pachetului java.net. Clasa ServerSocket implementează partea de server, creând un socket care ascultă pe un port specific și acceptă conexiuni de la clienți prin metoda accept(). Clasa Socket implementează partea de client, stabilind o conexiune către un server la o adresă IP și un port specificate. Odată conexiunea stabilită, comunicarea bidirecțională se realizează prin fluxuri de intrare (InputStream) și de ieșire (OutputStream), care permit transmiterea și recepția de date în mod continuu.

Arhitectura cu multiple ServerSocket-uri. O decizie arhitecturală importantă în proiectul Control Bot este utilizarea a patru instanțe ServerSocket distincte pe partea de server: unul pentru conexiunea

principală (autentificarea și stabilirea sesiunii) și câte unul pentru fiecare mod de control (Text, Mouse, Keyboard). Această abordare, deși pare redundantă la prima vedere, este justificată de considerente de performanță: separarea fluxurilor de date pe canale independente elimină necesitatea unui mecanism complex de multiplexare a mesajelor și reduce latența de procesare, fiecare mod de control având propriul său flux de comunicare dedicat. Această arhitectură este deosebit de benefică în rețeelele cu lățime de bandă redusă, unde dimensiunea mesajelor și timpul de procesare sunt factori critici.

III. Descrierea părții practice

III.1 Crearea aplicației pentru Android

Aplicația pentru Android reprezintă componenta client a sistemului Control Bot și funcționează ca o telecomedă digitală prin intermediul căreia utilizatorul poate controla de la distanță tastatura și mouse-ul unui calculator pe care rulează componenta de server. Aplicația este structurată în cinci activități distincte, fiecare cu responsabilități bine definite, și utilizează un model de comunicare bazat pe socket-uri TCP pentru transmiterea comenzilor către server. Navigarea între activități urmează un flux ierarhic: de la ecranul de conectare, utilizatorul accesează ecranul de selecție a modului de control, de unde poate alege unul dintre cele trei moduri de operare — Text, Mouse sau Keyboard.

III.1.1 Interfața principală (conectare)

Interfața principală, denumită și activitatea de conectare, constituie punctul de intrare al aplicației Control Bot. Rolul său fundamental este de a permite utilizatorului să stabilească o conexiune TCP cu serverul care rulează pe calculatorul gazdă. Această activitate reflectă paradigma standard a ecranelor de autentificare din aplicațiile moderne, adaptată însă la specificul comunicației în rețea locală.

Elementele interfeței. În zona centrală a ecranului sunt dispuse două câmpuri de introducere a datelor (EditText), aliniate orizontal unul lângă altul: primul câmp este destinat introducerii adresei IP a calculatorului gazdă, iar al doilea câmp este destinat specificării portului pe care serverul ascultă conexiuni. Sub aceste câmpuri, tot în zona centrală, este plasat un buton cu textul „Connect”, a cărui acționare declanșează procedura de conectare la server. În partea inferioară a ecranului se regăsește un text de tip link, „Already connected?”, care oferă o cale de acces rapid către ecranul de selecție a modului de control, pentru situațiile în care utilizatorul este deja conectat la server.

Validarea datelor de intrare. Înainte de inițierea conexiunii, aplicația efectuează o serie de validări asupra datelor introduse. Dacă ambele câmpuri sunt goale, se afișează un mesaj de tip Toast cu textul „Put an ip address and a port”. Dacă valoarea portului introdus nu se încadrează în intervalul valid (0–65535), se afișează un Toast cu mesajul „Put a value between 0 and 65535”. Câmpul destinat portului este configurat pentru a accepta exclusiv caractere numerice, respingând automat orice

introducere de litere sau simboluri. Aceste validări preventive asigură că doar date de intrare corecte sunt transmise către stiva de rețea, evitând astfel excepții la nivel de socket.

Procedura de conectare. La apăsarea butonului „Connect”, aplicația inițiază o cerere de conexiune TCP către adresa IP și portul specificate. În timpul procesului de conectare, se afișează un Toast informativ cu mesajul „Trying to connect...”, care notifică utilizatorul că operațiunea este în curs de execuție. În cazul în care conexiunea eșuează — fie din cauza unei adrese IP incorecte, fie din cauza indisponibilității serverului — se afișează un Toast de eroare cu mesajul „Failed to connect. Incorrect Ip address or port”. Dacă conexiunea reușește, aplicația navighează automat către activitatea de selecție a modului de control.

Accesul rapid „Already connected?”. Textul „Already connected?” din partea inferioară a ecranului este conceput pentru situațiile în care utilizatorul a efectuat anterior conectarea la server, dar a revenit din greșeală la ecranul principal. La apăsarea acestui link, aplicația verifică starea conexiunii prin intermediul clasei SocketManager: dacă socket-ul este activ și conexiunea este intactă, navighează direct la ecranul de selecție a modului de control, omițând pasul de reconectare; în caz contrar, afișează un Toast cu mesajul „Must connect first”.

Design responsiv. Pentru a asigura o aliniere proporțională a elementelor indiferent de dimensiunea ecranului dispozitivului, layout-ul utilizează un sistem de poziționare bazat pe procente și grupări cu diverse tipuri de aranjament (LinearLayout, TableLayout, ConstraintLayout). Această abordare garantează că interfața își păstrează aspectul și funcționalitatea atât pe ecrane mici, cât și pe ecrane de dimensiuni mai mari.

Clasa SocketManager. Conectarea la server este gestionată de o clasă statică denumită SocketManager, care încapsulează obiectul Socket și fluxurile de comunicare asociate acestuia. Caracterul static al clasei permite accesarea și modificarea variabilelor de conexiune din orice activitate și în orice moment al ciclului de viață al aplicației, fără a fi necesară inițializarea unui nou obiect la fiecare tranziție între activități. Acest model de design asigură continuitatea conexiunii pe parcursul întregii sesiuni de utilizare.

III.1.2 Interfața alegerii modului de control

Această activitate funcționează ca un hub central de navigare, oferind utilizatorului posibilitatea de a selecta modul de control dorit după ce conexiunea cu serverul a fost stabilită cu succes. Designul său este minimalist și funcțional, concentrând atenția utilizatorului asupra celor trei opțiuni principale.

Elementele interfeței. În zona centrală a ecranului sunt dispuse trei butoane de dimensiuni egale, aliniate vertical, centrate în mijlocul ecranului. Fiecare buton poartă denumirea unui mod de control: „Keyboard”, „Text” și, respectiv, „Mouse”. În colțul superior stâng al ecranului se regăsește un buton de navigare reprezentat printr-o săgeată orientată spre stânga, cu funcția de întoarcere la activitatea anterioară, adică la interfața principală de conectare.

Navigarea. Acționarea fiecărui buton declanșează tranziția către activitatea corespunzătoare: butonul „Keyboard” deschide interfața de emulare a tastaturii, butonul „Text” deschide interfața de introducere a textului, iar butonul „Mouse” deschide interfața de control al mouse-ului. Această organizare ierarhică a navigării asigură un flux intuitiv și previzibil, în care utilizatorul poate comuta cu ușurință între diferitele moduri de control și poate reveni oricând la ecranele anterioare. Butonul de „back” asigură navigarea către ecranul de conectare, permițând utilizatorului să reconfigureze parametrii conexiunii dacă este necesar.

III.1.3 Interfața „Text”

Interfața „Text” oferă utilizatorului posibilitatea de a transmite texte complete către calculatorul gazdă, texte care sunt introduse la tastatura acestuia ca și cum ar fi fost tastate manual de utilizator. Această abordare este semnificativ mai eficientă decât introducerea caracter cu caracter, permițând redactarea rapidă de cuvinte, propoziții sau paragrafe întregi.

Elementele interfeței. Interfața cuprinde următoarele componente: un buton de navigare cu funcția „back” în colțul superior stâng; un câmp de introducere a textului (EditText), denumit „text”, situat în zona centrală a ecranului; deasupra acestui câmp, în partea dreaptă, un buton etichetat „enter”, prevăzut cu simbolul tastei Enter; sub câmpul de text, centrat orizontal, un buton principal denumit „send”; sub butonul „send”, două butoane alăturate centrate orizontal — „delete last text” (în stânga) și „Backspace” (în dreapta).

Funcționalitatea elementelor. Câmpul „text” permite utilizatorului să compună textul dorit, care ulterior va fi transmis calculatorului. La apăsarea butonului „send”, conținutul câmpului este trimis către server prin intermediul clasei TextCommunication, iar serverul emulează tastarea acestui text la tastatura calculatorului, caracter cu caracter, producând efectul unei introduceri manuale. Butonul „enter” declanșează emularea tastei Enter pe calculatorul gazdă, util pentru confirmarea unei acțiuni sau trecerea la o linie nouă. Butonul „delete last text” calculează lungimea ultimului text transmis și emulează apăsarea tastei Backspace de un număr de ori egal cu numărul de caractere din acel text, realizând astfel o ștergere selectivă a ultimei intrări. Butonul „Backspace” emulează o singură apăsare a tastei Backspace, permițând ștergerea caracter cu caracter.

Clasa TextCommunication. Comunicarea specifică modului Text cu serverul este gestionată de o clasă statică dedicată, denumită TextCommunication, separată de SocketManager. Această separare arhitecturală asigură o organizare clară a responsabilităților: SocketManager gestionează conexiunea de bază (socket-ul principal), în timp ce TextCommunication gestionează fluxul de date specific modului Text, utilizând un canal de comunicare dedicat (un socket separat pe un port distinct). Această decizie permite procesarea independentă a mesajelor de tip text, fără interferențe cu celelalte canale de comunicare.

III.1.4 Interfața „Mouse”

Interfața „Mouse” este cea mai complexă dintre cele trei interfețe de control, oferind utilizatorului un control granular asupra cursorului mouse-ului și al butoanelor acestuia pe calculatorul gazdă. Designul său integrează atât funcții de navigare (mișcare, scroll), cât și funcții de acțiune (click stânga, click dreapta, hold), structurându-le într-un layout ergonomic care maximizează accesibilitatea și eficiența utilizării.

Elementele interfeței. Interfața este structurată pe mai multe rânduri orizontale. Rândul 1 conține butonul „back” în colțul superior stâng. Rândul 2 include două butoane alăturate, centrate orizontal — „scroll sus” (simbol: mouse cu săgeată în sus) și „scroll jos” (simbol: mouse cu săgeată în jos). Rândul 3 conține butonul „click stânga” (simbol: mouse cu butonul stâng colorat în negru), ansamblul „viteza mouse” (o casetă numerică cu valori între 1 și 200, prevăzută cu butoane de incrementare/decrementare) și butonul „click dreapta” (simbol: mouse cu butonul drept colorat în negru). Rândul 4 găzduiește un comutator de tip switch cu două poziții — „click” (modul de apăsare și eliberare instantanee) și „hold” (modul de menținere apăsată). Rândul 5 conține un asamblu de cinci butoane dispuse în formă de cruce (plus), cu un buton central dezactivat (simbol: cerc negru) și patru butoane direcționale reprezentate prin săgeți care indică direcția de mișcare.

Funcționalitatea elementelor. Butonul „scroll sus” declanșează o singură unitate de scroll în sens ascendent pe calculatorul gazdă, iar butonul „scroll jos” declanșează o singură unitate de scroll în sens descendent. Butonul „click stânga” funcționează în două moduri, determinate de starea comutatorului: în modul „click”, emulează o apăsare și eliberare instantanee a butonului stâng al mouse-ului; în modul „hold”, emulează menținerea apăsată a butonului stâng până la o nouă apăsare care eliberează tasta. Același comportament se aplică butonului „click dreapta”. Asamblul „mișcare mouse” mută cursorul pe calculator în direcția indicată de butonul apăsat, cu un număr de pixeli egal cu valoarea specificată în caseta „viteza mouse”. De exemplu, dacă valoarea este setată la 10, apăsarea butonului cu săgeată în dreapta va deplasa cursorul cu 10 pixeli spre dreapta. Această mecanism oferă un control precis asupra

vitezei de deplasare a cursorului, permițând atât mișcări fine (valori mici), cât și mișcări rapide (valori mari).

III.1.5 Interfața „Keyboard”

Interfața „Keyboard” este proiectată pentru a emula funcționalitatea completă a unei tastaturi fizice, oferind utilizatorului un control exhaustiv asupra caracterelor alfanumerice, tastelor de modificare, tastelor funcționale și comenzilor de navigare. Datorită complexității funcționale pe care trebuie să o acopere, aceasta este cea mai elaborată interfață a aplicației, cu o arhitectură vizuală structurată ergonomic.

Zona de navigare și control global. În extremitatea stângă-sus se regăsește butonul de navigare cu funcția „back”, reprezentat printr-o săgeată, care asigură întoarcerea la interfața de selecție a modului de control. În zona superioară centrală este implementat butonul „Release all”, cu rolul de a elibera simultan orice tastă rămasă blocată în starea activă (de exemplu, un Shift sau Ctrl menținut apăsat accidental), prevenind astfel comportamente nedorite ale calculatorului gazdă. Imediat sub acest buton, un comutator bidirecțional permite alternarea între două comportamente de acționare: „Hold” (menținerea apăsată a tastei selectate) și „Click” (apăsare și eliberare instantanee).

Tastele de modificare și de sistem. Arhitectura laterală este dedicată tastelor de modificare, sistem și execuție. Pe flancul stâng sunt dispuse vertical butoanele pentru comenzile „Esc”, „Tab”, „Caps Lock”, „Shift” și „Ctrl”. În mod complementar, flancul drept găzduiește butoanele „Backspace”, „Enter”, alături de comenzi duplicate pentru „Shift” și „Ctrl”, facilitând o dinamică de utilizare bi-manuală, similară cu dispunerea standard a unei tastaturi fizice PC. Aceste taste de modificare pot fi combinate cu alte taste, permițând emiterea de comenzi complexe precum Ctrl+C (copiere), Alt+Tab (comutare între ferestre) sau Shift+literă (introducerea majusculelor).

Modulele de selecție centrală. Axul central al interfeței optimizează spațiul vizual și este dedicat selecției de caractere și comenzi specifice. Acesta este divizat în trei module secvențiale, bazate pe liste derulante (dropdown / spinner): „Function buttons”, destinat selecției tastelor funcționale (F1–F12); „Letters and Numbers”, pentru caracterele alfanumerice de bază; și „Symbols”, pentru caractere speciale și de punctuație. Fiecare meniu derulant este acompaniat de un buton de execuție dedicat, denumit „Press”, a cărui acționare validează și transmite către server comanda corespunzătoare elementului selectat curent în listă.

Taste de bază și navigare. Sub modulele de selecție centrală este poziționat un rând orizontal care integrează tastele de bază și de sistem: butonul cu logo-ul Windows, „Alt”, „Space” (bara de spațiu) și „Alt Gr”. În secțiunea inferioară a ecranului este dispus un ansamblu direcțional compus din

patru butoane reprezentate prin săgeți (sus, stânga, jos, dreapta), organizate sub forma unui sistem de navigare clasic, esențial pentru deplasarea rapidă și precisă a cursorului de text sau navigarea în interfața grafică a calculatorului gazdă.

III.1.6 Comunicarea între telefon și calculator

Comunicarea între cele două componente ale sistemului Control Bot reprezintă nucleul funcțional al întregului proiect și se bazează pe o arhitectură de rețea bazată pe socket-uri TCP, cu o strategie de partiționare a canalelor de comunicare care optimizează performanța în condiții de rețea variabile.

Arhitectura cu patru ServerSocket-uri. La inițierea comunicării, aplicația de server de pe calculator creează un ServerSocket principal, dedicat acceptării noilor conexiuni de la clienți (telefoanele Android). Imediat după ce primul client se conectează cu succes la acest server principal, aplicația inițializează încă trei ServerSocket-uri, fiecare destinat exclusiv unuia dintre cele trei moduri de control: Text, Mouse și, respectiv, Keyboard. Această arhitectură cu patru canale independente de comunicare a fost adoptată din considerente de performanță și eficiență, având în vedere că aplicația este destinată și utilizării în medii cu viteză redusă a rețelei (de exemplu, rețele Wi-Fi congestionate sau conexiuni cu latență ridicată).

Avantajul separării canalelor. Fără partiționarea fluxurilor de date, toate mesajele de la cele trei moduri de control ar trebui să tranziteze un singur canal, necesitând un mecanism de multiplexare — adică un protocol de marcare a fiecărui mesaj cu identificatorul modului de proveniență, urmat de demultiplexarea pe partea de server. Acest proces ar introduce o complexitate suplimentară în cod și o latență adițională în procesarea fiecărui mesaj. Prin separarea canalelor, fiecare flux de date este procesat independent, fără necesitatea interpretării header-elor de multiplexare, iar dimensiunea mesajelor este minimizată, conținând exclusiv comanda relevantă.

Clase statice de comunicare. Atât pe partea de Android, cât și pe partea de Windows, toate clasele responsabile cu comunicarea prin rețea sunt implementate ca clase statice. Această decizie de design asigură persistența stării conexiunilor pe parcursul întregii sesiuni de utilizare, eliminând necesitatea serializării și deserializării obiectelor de comunicare la fiecare tranziție între activități. Clasa SocketManager gestionează conexiunea principală, în timp ce clasele specializate (TextCommunication, MouseCommunication, KeyboardCommunication) gestionează fluxurile de date specifice fiecărui mod de control, fiecare comunicând prin intermediul propriului socket dedicat.

III.2 Crearea aplicației pentru Windows

Componenta de server a sistemului Control Bot este o aplicație destinată sistemelor de operare Windows 10 și Windows 11, dezvoltată tot în limbajul de programare Java, care îndeplinește un rol dual: pe de o parte, funcționează ca server de rețea, acceptând conexiuni de la clienții Android și procesând comenzile primite; pe de altă parte, acționează ca emulator de dispozitive de intrare, traducând comenzile recepționate în acțiuni concrete de tastare și de mișcare a mouse-ului pe calculatorul gazdă.

Arhitectura fără interfață grafică. Aplicația pentru Windows este concepută să ruleze exclusiv în modul consolă (terminal), fără a dispune de o interfață grafică proprie. Această decizie de design este deliberată și justificată de natura funcțională a programului: aplicația operează în fundal, ca un proces de tip serviciu, al cărui unic scop este de a asculta comenzile de la clientul Android și de a le executa pe calculatorul local. Eliminarea interfeței grafice reduce consumul de resurse, simplifică arhitectura aplicației și permite rularea sa neîntreruptă, fără a interfera cu activitatea utilizatorului pe calculatorul gazdă.

Deși lipsa interfeței grafice poate părea o limitare, această abordare este comună în dezvoltarea de aplicații de tip server și demonstrează un principiu fundamental de design software: separarea preocupărilor (separation of concerns). Serverul nu necesită interacțiune umană directă, ci funcționează ca un serviciu care primește comenzi și le execută. Toate informațiile relevante (starea conexiunii, erori, mesaje de sistem) sunt afișate în consolă, oferind administratorului posibilitatea de a monitoriza funcționarea aplicației fără a necesita o interfață grafică complexă.

Clasa Robot — emularea dispozitivelor de intrare. Funcționalitatea centrală a aplicației de server se bazează pe clasa java.awt.Robot, furnizată de biblioteca standard Java. Această clasă oferă API-uri pentru generarea programatică a evenimentelor de intrare native ale sistemului de operare, inclusiv apăsarea și eliberarea tastelor, mișcarea cursorului mouse-ului, click-uri (stânga, dreapta) și acțiuni de scroll. Metoda keyPress(int keycode) simulează apăsarea unei taste, iar keyRelease(int keycode) simulează eliberarea acesteia. În mod similar, mouseMove(int x, int y) deplasează cursorul la coordonatele specificate, mousePress(int buttons) și mouseRelease(int buttons) simulează apăsarea și eliberarea butoanelor mouse-ului, iar mouseWheel(int wheelAmt) generează evenimente de scroll. Toate codurile de taste utilizate corespund constantelor definite în clasa java.awt.event.KeyEvent, asigurând astfel compatibilitatea cu dispunerea standard a tastaturii.

Gestionarea concurentă a conexiunilor. Aplicația de server gestionează simultan patru canale de comunicare, fiecare deservit de un fir de execuție (thread) dedicat. La pornirea programului, serverul

principal creează un ServerSocket pe un port configurabil, care așteaptă conexiunea inițială a clientului Android. După stabilirea acestei conexiuni, serverul inițializează cele trei ServerSocket-uri secundare, fiecare pe un port distinct, pentru cele trei moduri de control. Fiecare conexiune este gestionată de un thread independent, care citește în mod continuu mesajele de la client și le translatează în apeluri corespunzătoare ale clasei Robot. Această arhitectură concurentă asigură că mesajele de la diferite moduri de control sunt procesate în paralel, fără blocări reciproce, menținând astfel un timp de răspuns minim.

Bucla de citire și execuție. Pentru fiecare canal de comunicare, firul de execuție asociat operează într-o buclă infinită care efectuează următoarea secvență: citește un mesaj de la client prin fluxul de intrare al socket-ului; interpretează mesajul pentru a determina comanda solicitată (de exemplu, „KEY_PRESS A”, „MOUSE_MOVE 10 0”, „MOUSE_CLICK_LEFT”); invocă metoda corespunzătoare a clasei Robot; și reia bucla. Formatul mesajelor este deliberat simplu și compact, constând în identificatori textuali urmați de parametri numerici, pentru a minimiza dimensiunea datelor transmise pe rețea și a reduce latența de procesare.

Rularea în fundal și robustețe. Aplicația este concepută pentru a rula continuu pe parcursul sesiunii de utilizare, fără a necesita intervenția utilizatorului după configurarea inițială. În cazul în care conexiunea cu clientul este pierdută (de exemplu, telefonul iese din raza de acoperire a rețelei Wi-Fi), serverul detectează închiderea socket-ului și reintră în starea de așteptare a unei noi conexiuni, fără a necesita repornirea programului. Această funcționalitate de auto-recuperare conferă aplicației o robustețe operațională esențială pentru o utilizare fluentă și neîntreruptă.

IV. Concluzii

IV.1 Realizări și rezultate obținute

Proiectul Control Bot a atins obiectivul principal propus la momentul definiției sale: realizarea unui sistem funcțional de control de la distanță al unui calculator prin intermediul unui dispozitiv Android, bazat pe comunicarea în rețea locală prin socket-uri TCP. Aplicația Android oferă trei moduri de control distincte și complet funcționale — Text, Mouse și Keyboard — fiecare integrând un set coerent de comenzi care acoperă majoritatea nevoilor de interacțiune cu un calculator la distanță. Modul Text permite transmiterea rapidă de texte complete, eliminând necesitatea tastării caracter cu caracter. Modul Mouse oferă un control granular asupra cursorului, incluzând mișcare direcțională cu viteză reglabilă, click stânga și dreapta cu suport pentru modul hold, precum și funcții de scroll. Modul Keyboard emulează exhaustiv funcționalitatea unei tastaturi fizice, incluzând taste alfanumerice, taste funcționale, taste de modificare cu suport pentru combinații și taste de navigare direcțională.

Componenta de server pentru Windows a demonstrat stabilitate operațională și capacitate de a gestiona concurent cele patru canale de comunicare, traducând în timp real comenzile primite în acțiuni native de intrare prin intermediul clasei java.awt.Robot. Arhitectura cu patru ServerSocket-uri separate și-a dovedit eficiența în menținerea unui timp de răspuns redus, chiar și în condiții de rețea cu latență variabilă. Testarea aplicației în scenarii reale — controlul prezentărilor multimedia, navigarea în browser-uri și editarea de texte — a confirmat utilitatea practică și fiabilitatea sistemului.

IV.2 Dificultăți întâmpinate și lecții învățate

Dezvoltarea proiectului Control Bot nu a fost lipsită de provocări semnificative, a căror rezolvare a constituit o sursă valoroasă de învățare. O primă dificultate majoră a fost gestionarea concurentă a mai multor conexiuni socket pe partea de server. Sincronizarea thread-urilor pentru a evita condițiile de cursă (race conditions) la accesarea resurselor partajate, cum ar fi instanța clasei Robot, a necesitat o înțelegere profundă a mecanismelor de excluziune mutuală și a blocurilor synchronized din Java. Această provocare a oferit ocazia de a aprofunda conceptele de programare concurentă și de a implementa pattern-uri corecte de sincronizare.

O altă dificultate a fost asigurarea compatibilității interfeței Android pe o gamă diversă de dispozitive, cu dimensiuni și rezoluții de ecran diferite. Utilizarea exclusivă a layout-urilor bazate pe procente și a grupărilor responsive a rezolvat parțial această problemă, însă testarea pe dispozitive fizice multiple a fost esențială pentru identificarea și corectarea anomaliilor de afișare. Această experiență a subliniat importanța testării pe dispozitive reale, nu doar pe emulatoare.

De asemenea, emularea tastelor cu modificatori (de exemplu, Ctrl+C, Alt+Tab) a prezentat o complexitate inerentă, deoarece necesită menținerea stării apăsată a tastei de modificare în timp ce se emulează apăsarea tastei secundare. Implementarea corectă a acestui mecanism a implicat gestionarea explicită a secvențelor keyPress/keyRelease și asigurarea eliberării tuturor tastelor de modificare la ieșirea din modul Keyboard, funcționalitate realizată prin butonul „Release all”. Această provocare a evidențiat necesitatea unui management riguros al stării tastelor de modificare, care să prevină blocarea acestora în starea activă.

IV.3 Perspective de dezvoltare ulterioară

Deși aplicația Control Bot în forma sa actuală își îndeplinește obiectivele funcționale, există multiple direcții de dezvoltare ulterioară care ar putea extinde semnificativ capabilitățile și utilitatea sistemului. O primă direcție o constituie implementarea comunicării criptate prin protocolul TLS/SSL, care ar asigura confidențialitatea și integritatea datelor transmise între client și server, esențial pentru utilizarea aplicației în rețele nesigure sau pe internet. O a doua direcție vizează adăugarea funcționalității de control al mediului desktop — de exemplu, comenzi pentru volum, redare/pauză media, capturi de ecran sau transfer de fișiere — care ar transforma aplicația dintr-o telecomedă de bază într-un instrument de administrare remote comprehensiv.

O altă extensie valoroasă ar fi dezvoltarea unei interfețe grafice minimaliste pentru componenta de server, care ar afișa starea conexiunilor, adresele IP ale clienților conectați și un jurnal al comenzilor executate, facilitând monitorizarea și depanarea. În plus, suportul pentru conexiuni simultane de la mai mulți clienți, cu un sistem de autentificare prin parolă, ar permite utilizarea aplicației în contexte educaționale sau de prezentare, unde mai mulți utilizatori ar putea controla colaborativ același calculator. Implementarea acestor extensii ar plasa Control Bot în categoria aplicațiilor de administrare remote de nivel profesional, cu un potențial real de utilizare în mediul productiv.

V. Bibliografie

1. Oracle Corporation, Java Platform, Standard Edition Documentation, Oracle, 2024. Disponibil la: https://docs.oracle.com/en/java/javase/
2. Google, Android Developers — Documentation, Google, 2024. Disponibil la: https://developer.android.com/docs
3. Oracle Corporation, Class Robot — Java AWT API, Oracle, 2024. Disponibil la: https://docs.oracle.com/javase/8/docs/api/java/awt/Robot.html
4. Oracle Corporation, Class ServerSocket — Java Net API, Oracle, 2024. Disponibil la: https://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html
5. JetBrains, IntelliJ IDEA Documentation, JetBrains, 2024. Disponibil la: https://www.jetbrains.com/idea/documentation/
6. Google, Android Studio — User Guide, Google, 2024. Disponibil la: https://developer.android.com/studio/intro
7. David J. Eck, “Introduction to Programming Using Java”, Version 7.0, August 2014